Universität Zürich[UZH] | ETH zürich

Open Source Silicon Neuron: from schematics to GDS

Thomas Pigni

Institute of Neuroinformatics
Neuromorphic Cognitive Systems Group

MSc. Semester Project

Supervisors: Dr. Giacomo Indiveri
Sep 2025 – Jan 2026

Abstract

The objective of this semester project was to redesign the neuron synapse circuit proposed by Soo Hyun Lee[1] , with the aim of obtaining a transistor-level implementation that is compatible with physical layout and fabrication constraints. The redesigned circuit was developed using open-source electronic design automation tools, specifically *xschem* for schematic and, in a future step, *klayout* for layout preparation.

A redesign phase was required because the original implementation presented critical limitations related to transistor sizing. The initial design relied on low-voltage MOSFETs from the IHP130 process design kit (PDK); however, in order to ensure correct circuit operation, the channel length of several transistors was increased up to 20 μm. Such dimensions exceed the practical limits of the fabrication process and are therefore unsuitable for a manufacturable layout. As a consequence, alternative transistor options had to be evaluated, and the circuit parameters were systematically retuned to meet both functional and technological constraints.

The final design employs high-voltage MOSFETs from the IHP130 PDK, which provide a wider and more controllable subthreshold operating region compared to their low-voltage counterparts. This characteristic makes them particularly well suited for the intended neuromorphic operating regime, while allowing the circuit to be realized with realistic device dimensions and prepared for subsequent layout implementation.

1 Open-source software

A preliminary step of the project consisted in setting up a reliable open-source electronic design automation (EDA) environment. Several approaches were initially evaluated, including the use of virtual machines and remote access to Linux servers. However, these solutions proved to be either inefficient in terms of usability and performance.

The most reliable and practical solution was found to be the use of the *IIC-OSIC-TOOLS* Docker container, which provides a fully configured environment including all required open-source tools and process design kits (PDKs). In particular, this container integrates schematic capture and layout tools such as *xschem* and *klayout*, along with the necessary simulation engines and technology files, enabling a rapid and reproducible setup.

1.1 Docker container

The *IIC-OSIC-TOOLS* Docker image has a size of approximately 20 GB. To avoid excessive use of local storage on the host machine, the Docker image was stored on an external solid-state drive (SSD) connected via USB Type-C. While this approach effectively reduces disk usage on the main system, it introduces a constraint: when Docker is configured to store images on an external drive, all existing Docker images must be migrated to the same location, and Docker will fail to start if the external SSD is not connected. In such cases, an error message similar to the one shown in Figure 1 (left) is displayed.

To modify the Docker image storage location, the external SSD must first be connected. The image location can then be changed by navigating to *Settings* → *Resources* → *Disk image location* within the Docker configuration menu. The configuration used in this project is illustrated in Figure 1 (right).

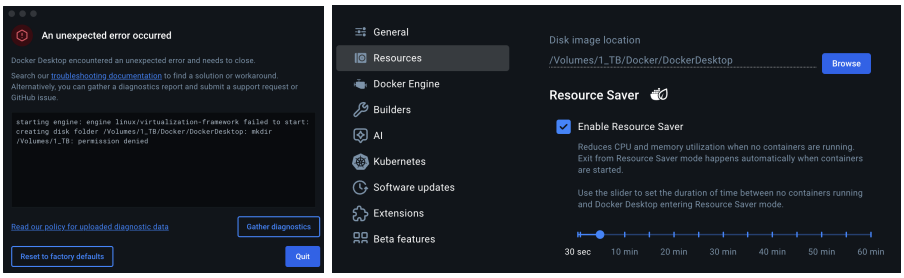


Figure 1: Error message displayed when Docker is started without the external SSD (left) and configuration of the Docker image storage location on an external drive (right).

After configuring the Docker image storage path, the next step consists in downloading the latest *IIC-OSIC-TOOLS* image and creating the corresponding container. This process is facilitated by a set of scripts provided in a GitHub

repository[1], curated by the Department for Integrated Circuits (ICD) at Johannes Kepler University (JKU). The repository includes pre-installed PDKs, which significantly simplifies the setup process and avoids issues encountered during manual PDK installation.

In addition to the core installation scripts, the repository provides multiple utilities for launching a graphical desktop session inside the Docker container. This feature is essential, as most EDA tools included in the environment require a graphical user interface and cannot be operated exclusively from the command line. Figure 2 (left) shows the file structure of the repository. On macOS systems, graphical access to the container can be achieved using *XQuartz*; in this case, the script *start_x.sh* was used to launch the container. Alternative scripts are available for other operating systems and display servers, such as VNC[2]. Figure 2 (right) shows the command-line interface of the running Docker container as displayed through *XQuartz*.

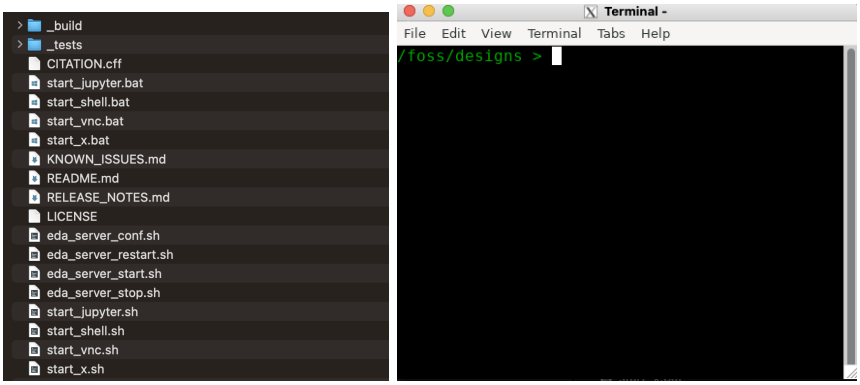


Figure 2: File structure of the *IIC-OSIC-TOOLS* GitHub repository (left) and command-line interface of the running Docker container accessed via *XQuartz* (right).

The initialization scripts provided in the repository also create a shared directory between the host system and the Docker container, which can be found at *$User/eda/designs*. This shared workspace allows design files to be easily transferred between the host environment and the container, facilitating efficient interaction with the open-source EDA tools installed within the Docker image.

1.2 Xschem

The open-source schematic editor *xschem* can be launched directly from the terminal by typing `xschem` and pressing `Enter`. It is strongly recommended

[1] Link: https://github.com/iic-jku/IIC-OSIC-TOOLS

[2] In preliminary tests on an Apple M1-based system, TigerVNC exhibited insufficient performance for practical use.

to start *xschem* from within the directory corresponding to the specific project (e.g., *./design/project1*), as the tool automatically generates a *simulation* subdirectory in the working directory. All netlists, simulation control files, and output data produced during circuit analysis are stored in this location.

When executed inside the *IIC-OSIC-TOOLS* Docker environment, *xschem* initially displays a startup page, shown in Figure 3. This page acts as a reference and provides a collection of commonly used schematic blocks, including elements for simulation setup, model inclusion, and library management for the IHP PDK. These blocks can be directly copied into a schematic, accelerating the creation of designs and reducing the likelihood of configuration errors.

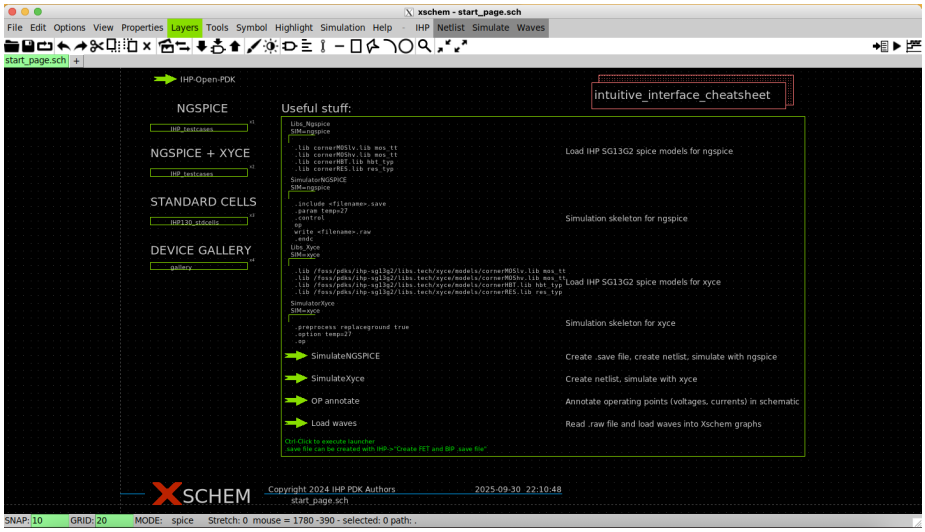


Figure 3: Xschem start page provided within the *IIC-OSIC-TOOLS* Docker environment.

The *xschem* graphical user interface provides a set of toolbar buttons that allow direct access to the most frequently used actions. These include schematic editing operations such as component insertion, wiring, copy and move commands, as well as functions for netlist generation and simulation execution. While most operations can also be performed through keyboard shortcuts, the graphical interface facilitates rapid prototyping, especially during the early stages of design. A concise list of the most commonly used commands, keyboard shortcuts, and practical tips for efficient schematic editing is provided in Appendix A and B.

Circuit simulations in *xschem* are performed using an external simulation engine, most commonly *NGSPICE*, although alternative engines are also supported. Simulation parameters, control statements, and model references are defined directly within the schematic by means of dedicated text and symbol blocks. This approach ensures that simulation settings are tightly coupled to the schematic itself: when a `.sch` file is transferred between different systems

or users, all associated simulation parameters are preserved.

Unlike some commercial EDA tools, *xschem* does not automatically resolve missing component libraries. Any symbol used in the schematic must explicitly reference an imported library; otherwise, the tool will report an error during netlist generation. This requirement does not apply to elements from the standard *xschem* library, such as passive components, power supplies, ground references, and ideal voltage or current sources, which are available by default. Proper management of library imports is therefore essential when working with technology-specific devices such as MOSFETs from a PDK.

An example of a minimal schematic including all elements required for device characterization is shown in Figure 4. In this representation, electrical connections between components are indicated by blue wires, which terminate at red square markers corresponding to the symbol pins. The green arrow connected to ground and labeled "Vds" represents an ammeter used to measure the drain–source current of the device under test. Device parameters such as channel width (w), channel length (l), number of fingers (ng), and multiplicity (m) are specified directly on the transistor symbol. On the right-hand side of the schematic, blocks responsible for simulation control and library inclusion are shown.

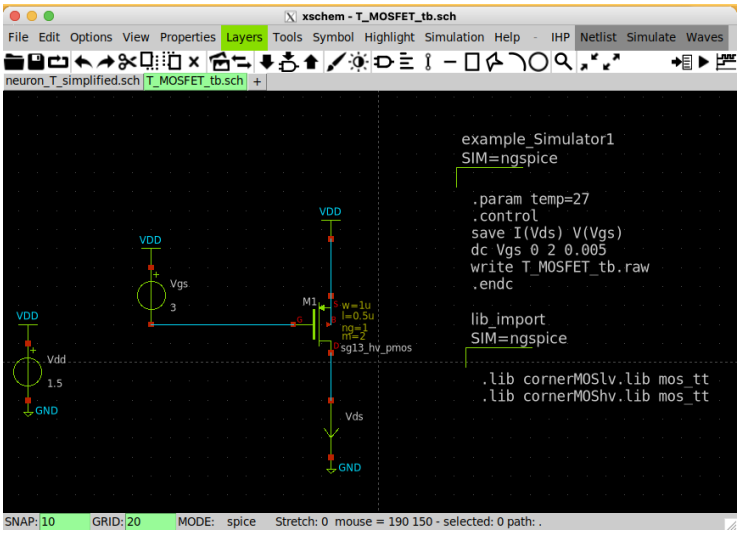


Figure 4: Example *xschem* schematic used for MOSFET characterization, including device parameters, simulation control blocks, and current measurement setup.

1.3 SPICE

Within this project, circuit behavior was analyzed primarily using two types of SPICE simulations: transient analysis and DC analysis. Each simulation

type serves a distinct purpose and provides complementary insight into circuit operation.

Transient simulations are employed to evaluate the time-domain evolution of node voltages and branch currents within the circuit. This type of analysis requires the definition of appropriate initial conditions and is particularly well suited for studying dynamic behaviors such as signal transitions, charge and discharge processes in capacitors, and in this case the event-driven responses of the neuron circuit. In contrast, DC simulations are used to characterize the static operating behavior of a circuit by sweeping one or more input voltages or currents and computing the corresponding steady-state operating points. A typical application of DC analysis in this work is the extraction of MOSFET characteristic curves, where the circuit is not evaluated as a function of time but rather as a function of the applied bias conditions.

Representative examples of transient and DC simulation setups are shown in Figure 5. The directive `.param temp=27` is used to define the operating temperature of the simulation in degrees Celsius, allowing the simulator to select the appropriate device models and temperature-dependent equations. Initial conditions for transient simulations are specified using the `.nodeset` statement, which provides starting values for selected node voltages and improves convergence during the initial operating point calculation.

The main differences between transient and DC simulations are specified within the `.control` block. This section defines which node voltages and branch currents are stored using the `save` command, as well as the simulation type and its corresponding parameters. For DC analysis, the `.control` block specifies the sweep variable, its range, and the sweep resolution. For transient analysis, it defines the total simulation time and the maximum allowable time step.

```
example_Simulator1
SIM=ngspice

.param temp=27

.nodeset v(vmem)=0 v(vref)=0 v(vreq)=0 v(vack)=1.5

.control

save V(vmem) I(Vin) I(Vcapcr) I(VmemFeed) I(Vidd)
tran 50n 3m
write neuron_T_simplified.raw
.endc
```

```
example_Simulator1
SIM=ngspice

.param temp=27
.control
save I(Vds) V(Vgs)
dc Vgs 0 2 0.005
write T_MOSFET_tb.raw
.endc

lib_import
SIM=ngspice

.lib cornerMOSlv.lib mos_tt
.lib cornerMOShv.lib mos_tt
```

Figure 5: Examples of SPICE simulation setups: transient analysis (left) and DC analysis (right).

In the case of a transient command such as `tran 50n 3m`, the simulator performs a time-domain analysis over a total duration of 3 ms, while enforcing a minimum time step of 50 ns. The selection of a sufficiently small minimum time step is essential to ensure numerical stability and convergence, particularly in circuits exhibiting fast transitions or sharp current spikes. It is important to note that this minimum time step is not applied uniformly over the entire simulation

interval. Instead, the *NGSPICE* transient solver automatically adapts the time step based on circuit activity: when signals vary slowly, the time step is increased to improve computational efficiency. On the oppositive, during rapid signal changes, it can be reduced down to the specified minimum limit to accurately capture the circuit dynamics.

1.4 Simulations

Once the schematic has been fully defined and all elements required for simulation have been properly configured, the simulation workflow in *xschem* consists of two main steps. First, the SPICE netlist is generated by clicking the `Netlist` button, which translates the schematic into a simulator-readable description. Subsequently, the simulation is launched using the `Simulate` button. If the setup is correct, a terminal window opens and displays the simulation log, including convergence information and potential warnings. Upon successful completion, the simulator generates a `.raw` output file, which is stored in the *simulation* directory associated with the project.

Although *xschem* provides basic plotting capabilities for visualizing simulation results, the analysis of waveforms in this project was primarily performed using an external Python-based workflow. The `.raw` simulation files produced by *NGSPICE* were imported into a Python notebook, where the data were processed and visualized using an interactive plotting library[3]. This approach offers greater flexibility and efficiency, particularly when dealing with multiple signals.

The *plotly* library was used for data visualization, as it enables interactive exploration of simulation results through features such as panning, zooming, and selective activation or deactivation of individual waveforms. These capabilities facilitate detailed inspection of circuit behavior and allow rapid comparison between different operating conditions or design iterations. Figure 6 shows an example of voltage waveforms plotted using this approach, including a zoomed view highlighting specific temporal features.

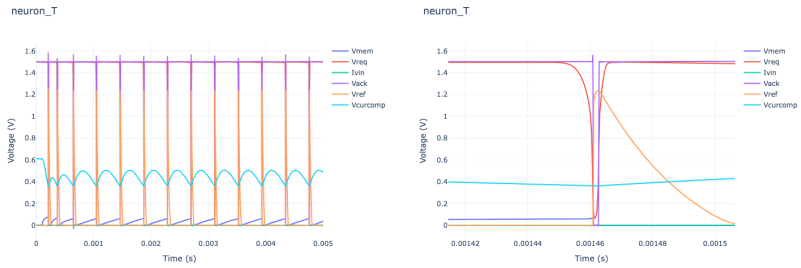


Figure 6: Example of voltage waveforms obtained from SPICE simulations and visualized using the *plotly* Python library.

[3] A minimal script for loading and plotting `.raw` simulation files is provided in the appendix.

2 Circuit: Introduction

The neuron circuit implemented in this project is largely derived from the design presented in the report by Soo Hyun Lee. As discussed previously, the primary modifications concern the choice and sizing of the transistors, while the overall circuit topology remain essentially unchanged. The circuit is based on a Differential Pair Integrator (DPI), which integrates an incoming spiking current in the nanoampere range and biases the membrane capacitor until a sufficient voltage level is reached to trigger neuronal firing. In this sense, the circuit stores the result of the temporal integration of the input current in the membrane capacitor, closely emulating the behavior of a biological neuron.

The neuron features an output signal `REQ`, and an input signal `ACK`, corresponding to the request and acknowledge signals, respectively. These signals are used to regulate the firing activity of the neuron. When the neuron reaches the firing condition, it asserts the `REQ` signal to indicate that it is ready to emit a spike. Upon receiving the `ACK` signal, the neuron resets the membrane capacitor and resumes the integration of the input current, thereby initiating a new integration cycle.

As shown in Figure 7, the overall neuron circuit can be partitioned into several distinct functional blocks. These include the DPI synapse, the membrane capacitor, a set of digital inverters responsible for generating and stabilizing the `REQ` and `ACK` signals, a positive feedback path between the membrane capacitor and the first inverter to accelerate spike initiation, a reset mechanism implementing a refractory period, and an adaptation block that modulates the firing behavior over longer time scales. Together, these blocks realize a leaky integrate-and-fire neuron with biologically inspired temporal dynamics.

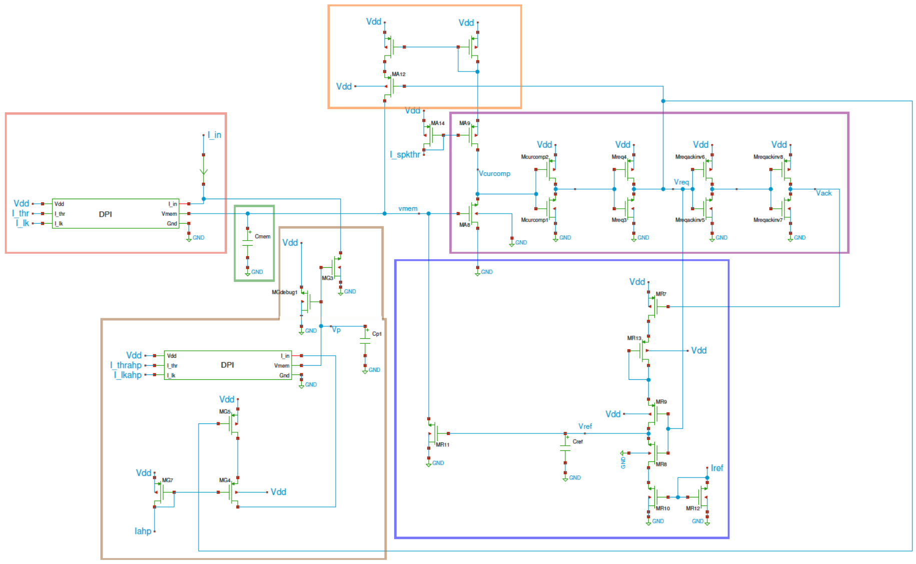


Figure 7: Schematic of the silicon neuron circuit originally proposed by Soo Hyun Lee. The DPI synapse is shown in red, the membrane capacitor in green, the digital inverter stages in purple, the positive feedback path in orange, the reset and refractory mechanism in blue, and the adaptation mechanism in brown.

3 Circuit: Devices characterization

3.1 MOSFET characterization

The IHP130 process design kit (PDK) provides two categories of MOSFET devices: low-voltage and high-voltage transistors. These two device types differ primarily in their fabrication stack, allowable supply voltage (V_{DD}), and threshold voltage (V_{th}). As a consequence, their electrical characteristics and suitability for specific operating regimes vary significantly.

A comparative evaluation of low-voltage and high-voltage devices was performed to determine the most appropriate transistor type for this project. Since the neuron circuit is intended to operate predominantly in the subthreshold regime, particular attention was given to the linearity and controllability of the drain current at low gate–source voltages. The high-voltage MOSFETs exhibit a wider and more gradual subthreshold region compared to their low-voltage counterparts, which makes them better suited for low-current neuromorphic applications. This behavior is accompanied by a higher threshold voltage and a requirement for a higher supply voltage; however, these trade-offs are acceptable within the design constraints of the circuit.

Figure 8 presents the transfer characteristics of both NMOS and PMOS high-voltage devices with channel width $W = 1\ \mu$m, channel length $L = 0.5\ \mu$m, and multiplicity $m = 2$. The drain current I_{D} is plotted as a function of the

gate–source voltage V_{GS} on a logarithmic scale, enabling a clear observation of subthreshold operation. Figure 9 shows the corresponding transfer characteristics for low-voltage devices with the same geometrical parameters.

The plots show a smooth exponential dependence of the drain current on V_{GS} in the subthreshold region for both NMOS and PMOS devices; however, the subthreshold operating range is wider for the high-voltage transistors compared to the low-voltage ones. For the high-voltage devices, the threshold voltage is approximately 0.6–0.7 V, whereas for the low-voltage devices it is around 0.4 V. Based on these results, high-voltage MOSFETs were selected for all subsequent circuit blocks in order to ensure stable and consistent subthreshold operation.

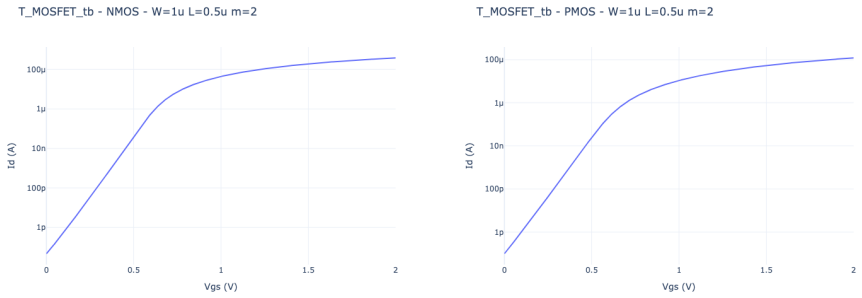


Figure 8: Testbench schematics and transfer characteristics of high-voltage NMOS and PMOS devices from the IHP130 PDK, simulated with $W = 1$ μm, $L = 0.5$ μm, $m = 2$, and $V_{\mathrm{DD}} = 1.5$ V.

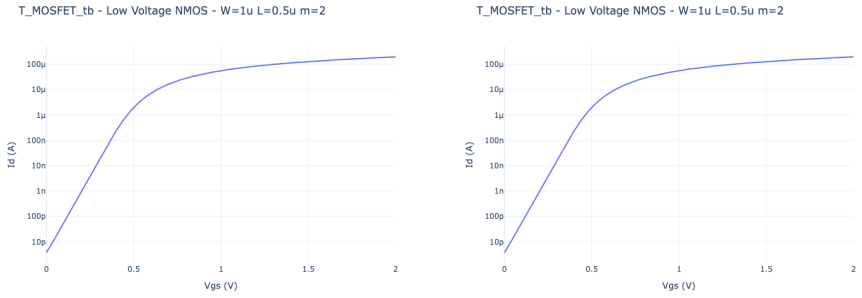


Figure 9: Testbench schematics and transfer characteristics of low-voltage NMOS and PMOS devices from the IHP130 PDK, simulated with $W = 1$ μm, $L = 0.5$ μm, $m = 2$, and $V_{\mathrm{DD}} = 1.5$ V.

3.2 Inverter characterization

Following the characterization of individual MOSFET devices, a CMOS inverter was analyzed, as it represents a fundamental building block of the neuron circuit. In particular, inverters are used to detect threshold crossings of the membrane voltage, generate digital control signals, and implement reset mechanisms. For these reasons, it is essential to characterize their switching behavior and current consumption in the operating regime of interest.

The inverter under study consists of a high-voltage PMOS and a high-voltage NMOS transistor, both configured with channel width $W = 1$ μm, channel length $L = 0.5$ μm, and multiplicity $m = 2$. The corresponding testbench schematic is shown in Figure 10. The input voltage V_{GS} is swept over the full operating range, while the output voltage and device currents are monitored.

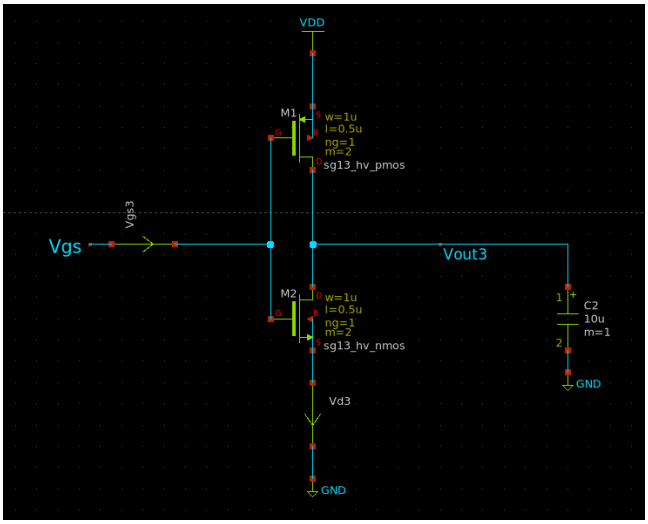


Figure 10: CMOS inverter testbench schematics.

Figure 11 illustrates the voltage transfer characteristic of the inverter. The output voltage exhibits a sharp transition between the logic-high and logic-low states, with a well-defined switching threshold located near the of the subthreshold region of the transistors. This behavior is particularly desirable for neuromorphic circuits, as it enables reliable detection of membrane voltage crossings while maintaining low static power consumption.

In addition to the voltage behavior, the drain currents of the NMOS and PMOS devices were analyzed as a function of the input voltage. The current plot shows a pronounced peak around the switching threshold, corresponding to the region where both transistors are partially conducting. Outside this transition region, the inverter operates predominantly in a low-current state, as either the NMOS or the PMOS device is turned off. This characteristic ensures that dynamic power consumption is predominantly associated with switching

events while the static current consumption remains relatively low.

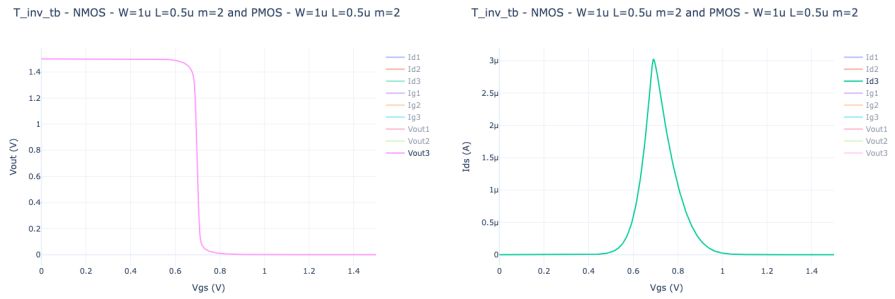


Figure 11: Voltage transfer characteristic (left) and drain current behavior as a function of the input voltage (right).

Overall, the inverter exhibits stable switching behavior, controlled current consumption, and a suitable threshold voltage for interaction with the membrane capacitor dynamics.

3.3 DPI characterization

The Differential Pair Integrator (DPI) is a fundamental building block of the neuron circuit and is responsible for converting an input current into a membrane voltage exhibiting biologically inspired temporal dynamics.

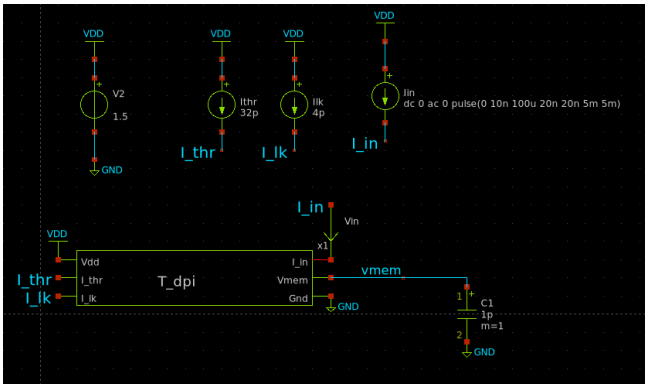


Figure 12: Testbench used for the characterization of the DPI block.

The schematic of the DPI employed in this work is shown in Figure 13. The input current I_{in} is injected into a differential pair and compared against a threshold bias current I_{thr}. The resulting current is mirrored and combined with the leak branch, which is biased by the current I_{lk}. The output node of the DPI is connected to the membrane capacitor C_{mem}, where the membrane voltage

V_{mem} is generated. The presence of the leak current introduces a leaky integration behavior, preventing unbounded voltage growth and ensuring a stable operating point.

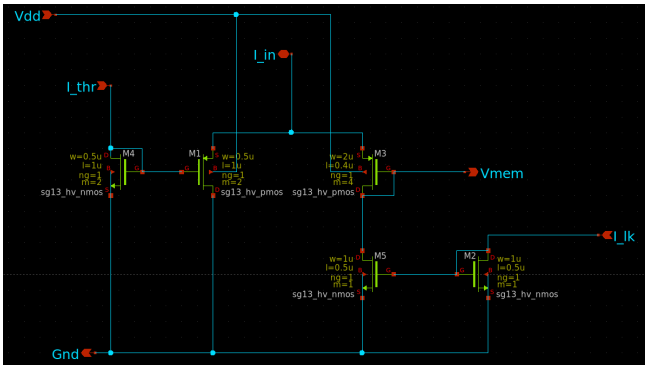


Figure 13: Schematic of the DPI block.

Figure 14 shows the same schematic with selected branch currents explicitly labeled in order to facilitate an analytical understanding of the circuit operation. Based on this representation, the following qualitative current relationships can be derived:

$$I_{\text{in}} = I_{D1} + I_{D3}, \qquad I_{D1} = I_{\text{thr}} \tag{1}$$

$$I_{D3} = I_{\text{mem}} + I_{D5}, \qquad I_{D5} = I_{\text{lk}} \tag{2}$$

$$I_{\text{mem}} = I_{\text{in}} - I_{\text{thr}} - I_{\text{lk}} \tag{3}$$

$$I_{\text{mem}} > 0 \quad \text{if} \quad I_{\text{in}} > I_{\text{thr}} \tag{4}$$

$$I_{\text{mem}} < 0 \quad \text{if} \quad I_{\text{in}} < I_{\text{thr}} \tag{5}$$

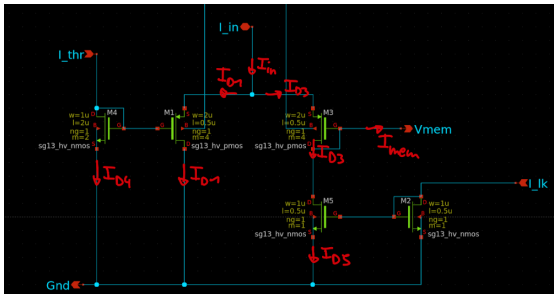


Figure 14: DPI schematic with labeled currents used for analytical evaluation.

From these expressions, it follows that a positive current flows into the membrane node when the input current exceeds the threshold current (assuming a negligible leak contribution). Conversely, when the input current is below threshold, the membrane node experiences a net discharging current. Independently of the input current value, the presence of the leak current continuously discharges the membrane capacitor, thereby enforcing a leaky integration behavior.

When a step or pulse of input current is applied, as shown in Figure 15, the membrane voltage V_{mem} increases following an exponential-like transient. The shape of this response is determined by the effective transconductance of the differential pair and by the value of the membrane capacitor. This behavior is clearly observed in the transient simulation results presented in Figure 16, where a sudden increase in I_{in} produces a gradual rise of V_{mem}. The initial fast response is associated with the differential pair operation, while the slower approach to steady state is governed by the leak mechanism.

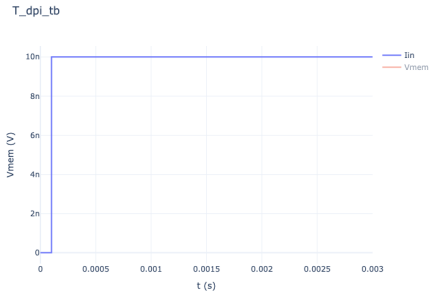


Figure 15: Input current I_{in} applied to the DPI.

The zoomed-in plots in Figure 16 further highlight the smooth integration dynamics at short time scales, confirming that the DPI responds continuously to input current variations without introducing abrupt voltage transitions. This property is essential for stable neuron operation, as it enables reliable comparison of the membrane voltage with the firing threshold implemented by the subsequent inverter stages.

Overall, the DPI provides a robust and tunable mechanism for current integration, with its temporal dynamics controlled by bias currents and capacitor values. These characteristics make it particularly suitable for neuromorphic neuron implementations, where accurate temporal integration and low power consumption are critical requirements.

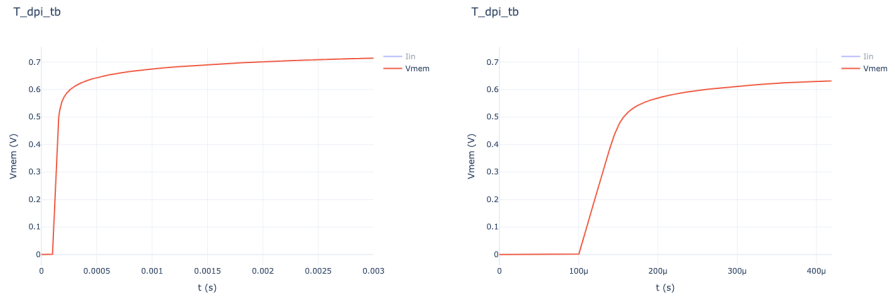


Figure 16: Membrane voltage response to the input current and zoomed-in transient behavior.

4 Circuit: Simulation

All circuit-level simulations were performed using a consistent set of bias currents, supply voltages, and timing parameters. A summary of the biasing and excitation conditions used throughout the simulations is shown in Figure 17.

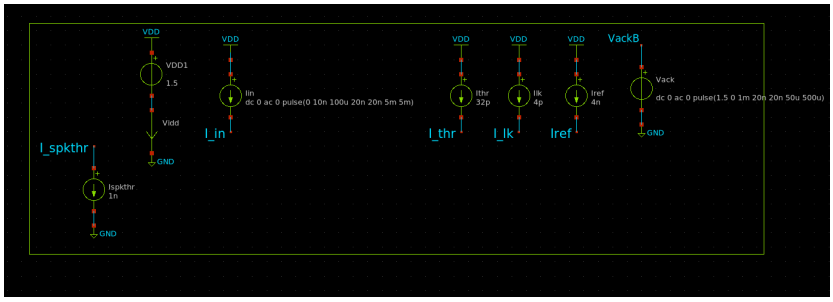


Figure 17: Bias currents and excitation parameters used for circuit-level simulations.

4.1 Simulation Without Adaptation

In order to isolate and analyze the core neuron dynamics, simulations were first carried out with the adaptation mechanism disabled. This allows the behavior of the membrane integration, spike initiation, and reset mechanisms to be studied independently of longer-term firing-rate modulation.

4.1.1 Request signal

When an input current is injected into the DPI block, it is integrated onto the membrane capacitor C_{mem}, causing the membrane voltage V_{mem} to increase over

time. As V_{mem} approaches the switching threshold of the first inverter stage, the circuit enters a strongly nonlinear operating region. At this point, a positive feedback mechanism is activated through the signal V_{req}, which significantly increases the charging rate of C_{mem}.

This positive feedback results in a rapid transition of the membrane voltage once the firing threshold is approached. Faster voltage transitions reduce the duration of switching events, which are the dominant contributors to dynamic power consumption, thereby improving the overall energy efficiency of the neuron.

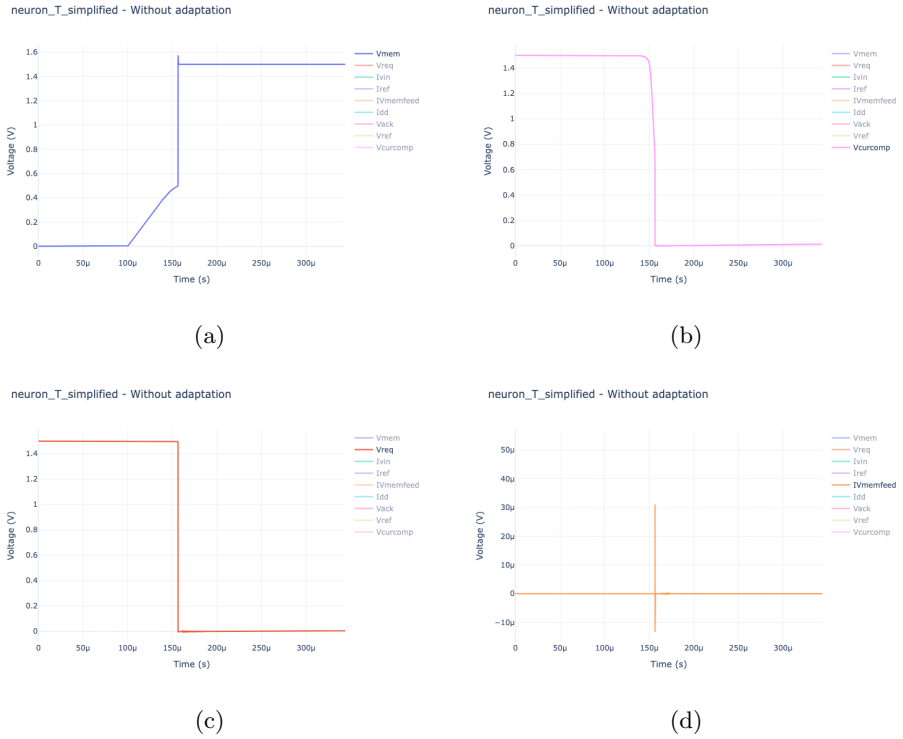


Figure 18: Request signal generation without adaptation. (a) Membrane voltage V_{mem}, (b) comparator-related voltage V_{curcomp}, (c) request signal V_{req}, and (d) membrane charging feedback current I_{Vmemfeed}.

The current comparator that would normally regulate the charging current of the membrane capacitor was initially removed to obtain a simplified circuit topology. However, at this stage of the analysis, the comparator is reintroduced into the circuit. The presence of this block influences the effective switching threshold of the first inverter and therefore affects the spike initiation dynamics.

As observed in the simulation results in Figure 18, the rise of V_{mem} is accompanied by a decrease in the comparator-related voltage V_{curcomp}. As a result,

both the PMOS and NMOS devices of the first inverter (MA9–MA8 in Figure 7) begin to conduct as the inverter transitions between logic states. The resulting short-circuit current is mirrored back into the membrane node, forming a positive feedback loop that further accelerates the rise of V_{mem} and sharpens the inverter switching behavior.

Consequently, the request signal V_{req} transitions to a low logic level, consistent with its active-low definition. At this point, the neuron asserts a request to fire and remains in this state until an acknowledge signal (`ACK`), which is also active-low, is received from the surrounding system.

4.1.2 Acknowledge signal

When an acknowledge signal (`ACK`) is asserted at $t = 1$ ms in Figure 19 (a), the neuron enters the reset and refractory phase. In this condition, the charge stored on the membrane capacitor C_{mem} is discharged to ground through transistor MR11. This transistor is enabled only when both V_{ack} and V_{req} are active, ensuring that the reset operation occurs exclusively after a valid firing request.

The conduction state of MR11 is governed by the voltage V_{ref}, which is stored on the capacitor C_r. During the acknowledge phase, C_r is discharged in a controlled manner through transistor MR8, whose drain current is regulated by the current comparator formed by transistors MR10–MR12 which can be seen in more detail in Figure 7. This configuration establishes a well-defined discharge profile for C_r, and consequently for V_{ref}.

This mechanism allows transistor MR11 to remain conductive until the voltage V_{ref} drops below the level required to sustain its operation. As long as MR11 is active, the membrane capacitor C_{mem} is prevented from recharging, thereby inhibiting the generation of a new firing request. This interval defines the refractory period of the neuron and is directly controlled by the reference current I_{ref}, which sets the discharge rate of C_r.

As shown in the simulation results of Figure 19, when the `ACK` signal is asserted, the membrane voltage V_{mem} is rapidly discharged to ground, indicating an effective reset of the neuron state. In contrast, the voltage V_{ref} decreases more gradually, reflecting the controlled discharge of the capacitor C_r. Once V_{ref} falls below the threshold required to keep transistor MR11 conductive, the reset path is disabled. At this point, V_{mem} begins to rise again as the input current is reintegrated, and the neuron becomes capable of generating a new firing request. The time required for C_r to discharge below the MR11 threshold defines the *refractory period* of the neuron and is controlled by the current I_{ref}.

4.1.3 Power consumption

The power consumption of the neuron circuit was evaluated by monitoring the supply current I_{DD}, which represents the current drawn from the supply voltage V_{DD} by the entire circuit. This analysis provides insight into both the static and dynamic power behavior of the neuron during operation.

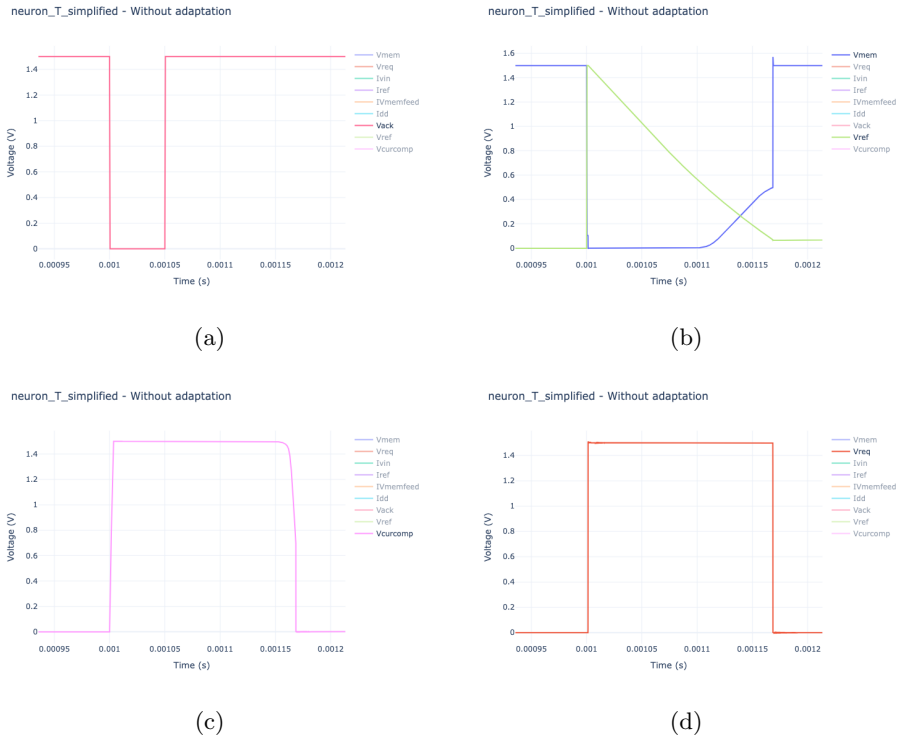


Figure 19: Acknowledge and reset behavior without adaptation. (a) Acknowledge signal V_{ack} (b) Membrane and reference voltages (V_{mem} and V_{ref}) (c) Comparator-related voltage V_{curcomp} (d) Request signal V_{reqB}.

As shown in the simulation results in Figure 20, the neuron exhibits a predominantly low baseline current during periods of membrane integration, reflecting subthreshold operation of the DPI and biasing circuitry. In contrast, pronounced current spikes are observed during `REQ` and `ACK` events. These spikes reach peak values of approximately 60 μA and correspond to moments when internal capacitors are rapidly charged or discharged and when inverter stages switch state.

The current associated with the capacitor C_r further highlights the contribution of the reset and refractory circuitry to the overall power consumption. During the acknowledge phase, transient currents arise from the controlled discharge of C_r, which sets the duration of the refractory period. Outside these events, the current returns to a low steady-state value.

Overall, the power consumption profile confirms the event-driven nature of the neuron circuit: significant current is drawn only during spike generation and reset operations, while the integration phase remains highly energy efficient. This behavior is consistent with the design objectives of neuromorphic systems,

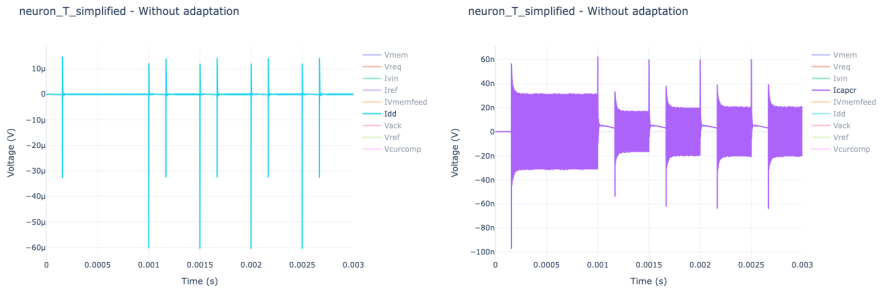


Figure 20: Power consumption without adaptation. Left: total supply current I_{DD} drawn from V_{DD}, showing transient current spikes during `REQ` and `ACK` events. Right: current associated with the refractory capacitor C_r, highlighting the contribution of the reset and refractory circuitry to dynamic power consumption.

where low average power consumption is achieved through sparse, event-based activity.

4.2 Simulation with Adaptation

During the implementation of the adaptation block, the feedback path realized through transistors M2–M3 and M20 in Figure 23 was also investigated. Several configurations were explored in order to identify a suitable set of parameters. The final transistor dimensions used for the complete schematic (including the adaptation block) are reported in the Appendix.

All the current parameters used for the simulations presented in this section are shown in Figure 21.

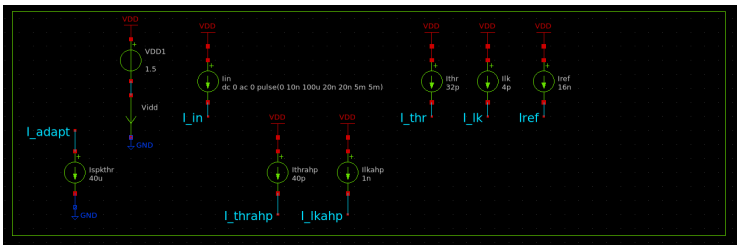


Figure 21: Current parameters used to bias the first and second DPI blocks, input current stimulus and the reference currents used for refractory and adaptation mechanisms.

4.2.1 Feedback Path

The feedback path required careful tuning of its transistor parameters and, in the explored configuration, it showed only a limited influence on the firing frequency. Potential low-power benefits of this feedback mechanism are therefore still under investigation.

Small current spikes of approximately 600–650 nA can be observed at the `Vmem` node. These spikes are not synchronized with the `REQ` signal but appear delayed by approximately 10 ns. This behavior is likely related to simulation time-step resolution and the fast transient response of the feedback transistors but it is not helping the first stage of the inverter to switch faster and thus saving energy.

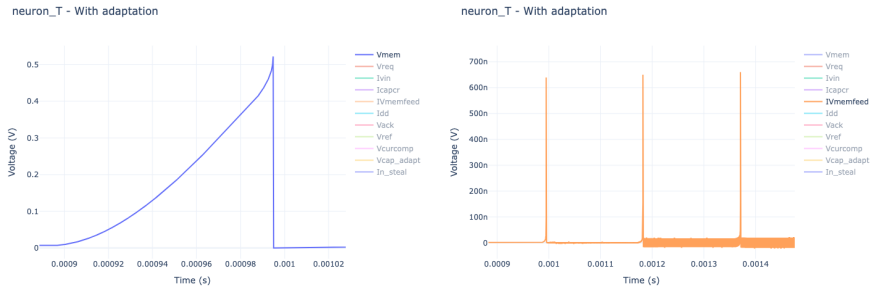


Figure 22: On the left: Zoom of the `Vmem` node potential. On the right: current injected into the `Vmem` node by the feedback path

It can also be observed that the `Vmem` node does not significantly charge during these short current spikes. Further investigation is required to identify an optimal set of parameters that would allow the feedback path to have a more pronounced effect.

4.2.2 Simulation Results

Figure 23 shows the complete circuit including the adaptation block and the feedback path. Compared to the original circuit, the explicit threshold current used to control neuron firing has been removed. This threshold current was found to be unnecessary and only introduced an additional tuning parameter during simulation.

The adaptation circuit operates as follows. Each `REQ` event activates transistor M19, which injects a constant current into a second DPI block. This second DPI is characterized by a different threshold and leakage current compared to the main DPI. These current values are reported in Figure 21.

The second DPI charges the node labeled `vcap_adapt` in Figure 23. The voltage at this node controls transistor M18, which diverts part of the input current away from the main DPI. As a result, the effective input current to the neuron is reduced, thereby lowering the firing rate.

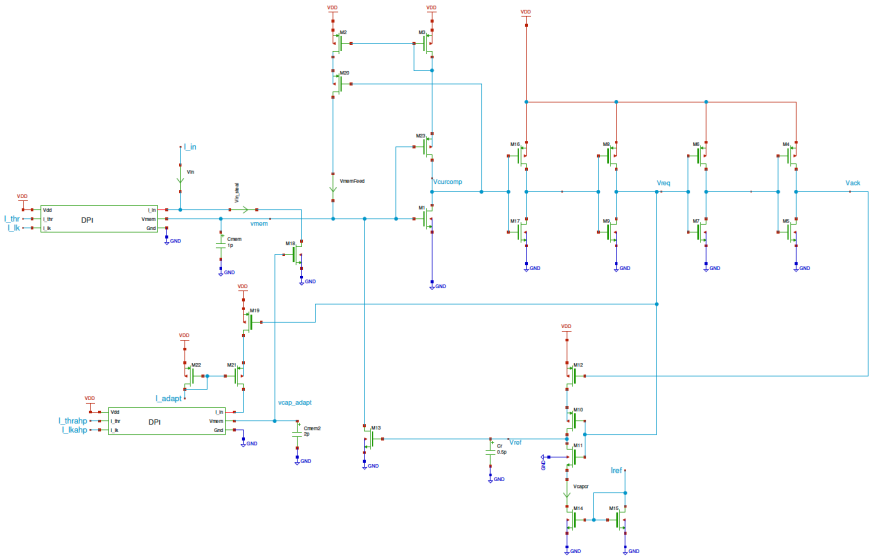


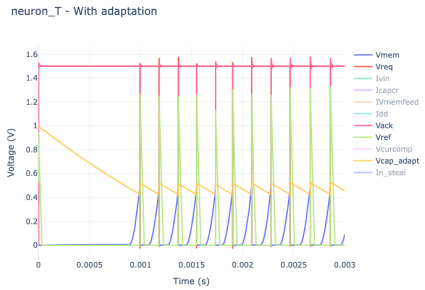
Figure 23: Complete neuron circuit with adaptation block and feedback path.

In other words, the adaptation mechanism introduces a negative feedback that limits the firing rate autonomously. If the firing rate becomes too high, frequent `REQ` signals cause `vcap_adapt` to increase, turning M18 on for a longer time and increasing the stolen current from the input. This reduces the current available to the main DPI and consequently decreases the firing frequency.

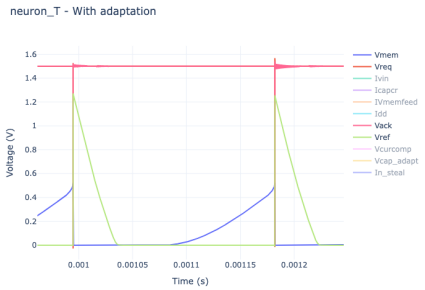
From Figure 24(b), it can be observed that the refractory period is no longer the dominant factor limiting the firing rate; instead, the adaptation block determines the maximum achievable frequency. Figure 24(a) shows that the adaptation voltage `vcap_adapt` does not return to zero but reaches a steady-state value, indicating that the neuron has reached its maximum firing frequency for the chosen adaptation current.

Figure 24(c) presents a detailed view of a single firing event. During this event, the `REQ` and `ACK` signals transition low, `Vref` increases to enforce the refractory period, and `Vmem` is discharged accordingly.

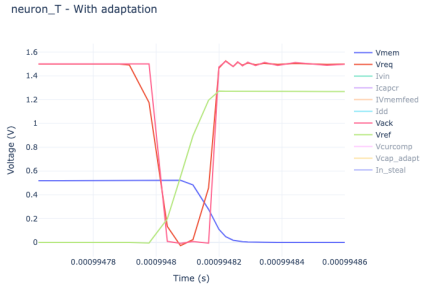
Finally, Figure 24(d) shows the input current (green) together with the diverted current (blue). Initially, the adaptation block diverts most of the input current. As `vcap_adapt` decreases, transistor M18 gradually turns off, allowing more current to flow into the main DPI. This behavior confirms that the adaptation mechanism effectively prevents the neuron from firing at excessively high rates.



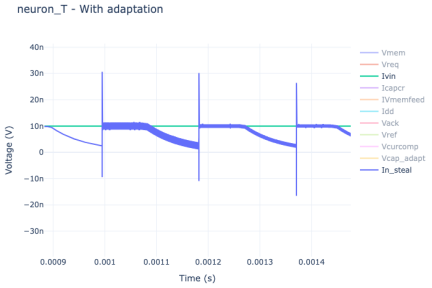
(a) Evolution of `Vmem`, `Vref`, `REQ`, `ACK`, and `vcap_adapt` during repeated firing events.



(b) Long-term behavior showing that the firing frequency is limited by the adaptation mechanism rather than by the refractory period.



(c) Zoomed view of a single firing event, highlighting the interaction between `REQ`, `ACK`, `Vref`, and the discharge of `Vmem`.



(d) Input current (green) and stolen current through M18 (blue), illustrating the effect of the adaptation block on the effective input current.

Figure 24: Simulation results of the neuron circuit with adaptation.

5 Layout: Mismatching techniques

After completing the circuit-level design and verification, the next critical step toward fabrication is the physical layout of the circuit. At this stage, device mismatches introduced by process variations, gradients, and parasitic effects can significantly degrade circuit performance, particularly in analog and neuromorphic systems operating in the subthreshold regime. For this reason, several well-established mismatch mitigation techniques were studied and are briefly discussed in this section, as they will be relevant during the layout phase of the design.

5.1 Common centroid technique

The common centroid technique is one of the most widely used layout strategies for reducing systematic mismatches caused by process gradients, such as variations in oxide thickness, dopant concentration, or temperature across the silicon die[2]. The core idea is to arrange matched devices symmetrically around a common geometric center so that linear gradients affect all devices equally and are therefore averaged out.

Figure 25: Example of the common centroid technique, where different letters denote different device types arranged symmetrically around a common center.

In practice, as illustrated in Figure 25, devices are divided into multiple identical unit transistors and placed in interleaved patterns such as ABBA or more complex symmetric arrangements. The gate, source, and drain connections are then routed symmetrically, often using multiple metal layers, in order to further reduce parasitic mismatches. This technique is particularly effective for layout-critical structures such as differential pairs and current mirrors, which play a central role in neuromorphic circuits.

Common centroid layouts are typically surrounded by substrate guard rings to reduce noise coupling and to mitigate latch-up susceptibility.

5.2 Multiple collector (guard ring) technique

In CMOS technologies, parasitic bipolar junction transistors (BJTs) are unavoidably formed due to the vertical and lateral doping profiles of the devices and the substrate[3]. These parasitic BJTs are not explicitly designed but arise naturally from the coexistence of p-type and n-type regions required for NMOS and PMOS transistors.

Specifically, as illustrated in Figure 26, an NMOS transistor implemented in a p-type substrate forms a parasitic NPN transistor, where the source or drain diffusion acts as the emitter, the p-type substrate acts as the base, and a neighboring n-type region or n-well acts as the collector. Similarly, a PMOS

transistor placed in an n-well forms a lateral parasitic PNP transistor. When NMOS and PMOS devices are placed in close proximity, these two parasitic BJTs can couple together to form a parasitic thyristor structure.

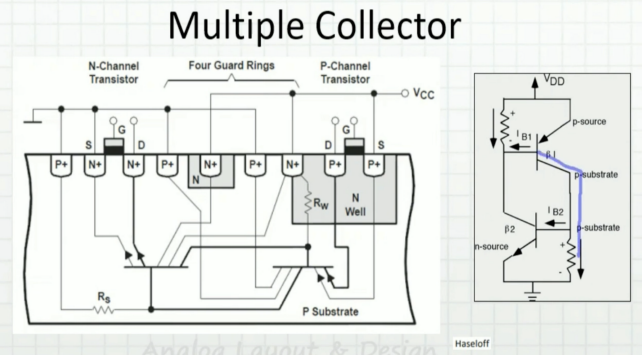


Figure 26: Cross-sectional view illustrating parasitic NPN and PNP bipolar junction transistors formed in a CMOS process and the use of multiple guard rings to collect substrate currents and reduce latch-up susceptibility.

Under normal operating conditions, these parasitic devices remain inactive because their base–emitter junctions are reverse biased, as the source and bulk terminals of the transistors are typically connected to ground or V_{DD}. However, transient substrate currents caused by switching activity, voltage overshoots, or electrostatic discharge events can locally raise the substrate or well potential. If this potential increase forward-biases the base–emitter junction of either parasitic BJT, the resulting collector current can activate the parasitic transistor. This feedback may latch the structure into a low-impedance conductive state between V_{DD} and ground, a condition known as latch-up.

Latch-up is problematic because it results in large static currents that persist even after the triggering event has ended, potentially leading to excessive power dissipation and permanent damage to the circuit.

The multiple collector (guard ring) technique addresses this issue by surrounding devices with heavily doped substrate or well contacts tied to a fixed potential. These guard rings provide low-resistance paths that collect injected substrate currents before they can significantly raise the local substrate voltage, thereby reducing the effective gain of the parasitic BJTs.

In addition to preventing latch-up, guard rings also suppress substrate noise propagation and help mitigate electromagnetic interference. This isolation is particularly important in subthreshold neuromorphic circuits, where small voltage or current disturbances can significantly affect circuit behavior.

5.3 ESD protection

Electrostatic discharge (ESD) events represent a significant reliability concern in fabricated integrated circuits. Sudden voltage spikes at input or output pads can

damage thin gate oxide layers or unintentionally activate parasitic structures, potentially leading to permanent circuit failure.

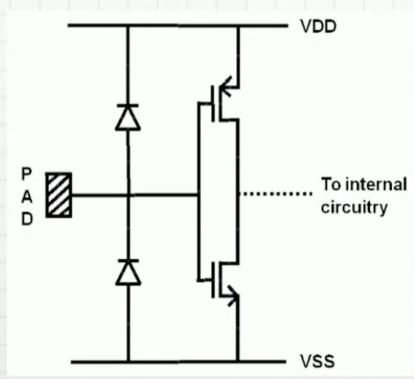


Figure 27: Simplified example of ESD protection using diode-connected devices to safely divert positive and negative voltage transients from an I/O pad to the supply rails.

A common ESD protection strategy consists of placing diode-connected devices between the signal pads and the supply rails[4] (V_{DD} and V_{SS}), as illustrated in Figure 27. In the presence of a positive voltage spike, the excess current is safely diverted to V_{DD}, while negative voltage spikes are shunted to ground. This clamping action limits the voltage excursion at the pad and protects the internal circuitry by preventing large voltage differentials across sensitive devices.

Although ESD protection structures are typically implemented at the pad-frame level, their presence and interaction with nearby circuitry must be considered during layout, particularly in compact or sensitive analog and neuromorphic designs.

5.4 N-well proximity effects

N-well proximity effects arise from non-uniform dopant distributions near the edges of n-wells[5]. During ion implantation and subsequent thermal diffusion steps, dopants can scatter and redistribute unevenly, leading to local variations in threshold voltage, body-effect coefficient, and carrier mobility for transistors placed close to well boundaries.

These effects can introduce systematic mismatches in otherwise identical devices. To mitigate N-well proximity effects, transistors should be placed at a sufficient distance from well edges, typically on the order of a few micrometers. Additional mitigation techniques include the use of dummy devices at the periphery of transistor arrays and maintaining symmetrical placement with respect to well boundaries.

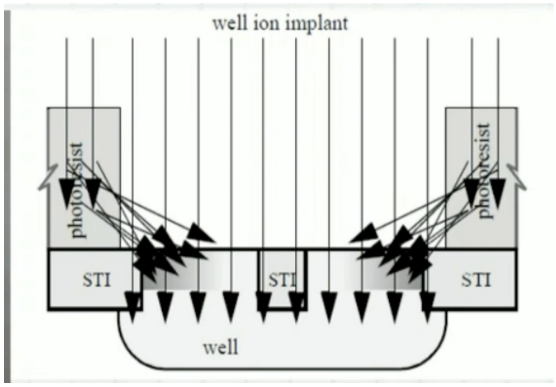


Figure 28: Illustration of N-well proximity effects showing non-uniform dopant distribution near well edges.

A Useful commands in *xschem*

This appendix summarizes frequently used keyboard shortcuts for efficient schematic editing in *xschem*.

- `Ctrl + s` — Save the current schematic
- `Ctrl + i` — Insert a symbol
- `Delete` — Delete selected objects (on macOS: `Fn + Backspace`)
- `Space` — Pan the view (particularly convenient when using a trackpad)
- `w` — Draw a wire
- `t` — Insert text annotations
- `q` — Edit properties of the selected object
- `c` — Copy selected objects
- `m` — Move selected objects
- `u` — Undo the last action
- `f` — Zoom to fit the schematic view
- `z` — Zoom into a selected region
- `R` — Rotate selected objects (90° clockwise)
- `F` — Flip selected objects

B Tips and practical considerations for *xschem*

- The function *Tools* > The *Align to Grid* function aligns schematic symbols and wires to the currently selected grid. This feature is particularly useful when opening schematics created with different grid settings.
- Schematics can be exported as PDF files using *File* > *Image export* > *PDF export* or *PDF export full*. The option *PDF export full* exports the complete schematic canvas, whereas *PDF export* exports only the currently visible region.
- When closing a schematic that contains unsaved modifications, *xschem* prompts the user to confirm whether the changes should be **discarded** without being saved. Care must be taken to avoid accidentally closing the schematic and discarding recent modifications.

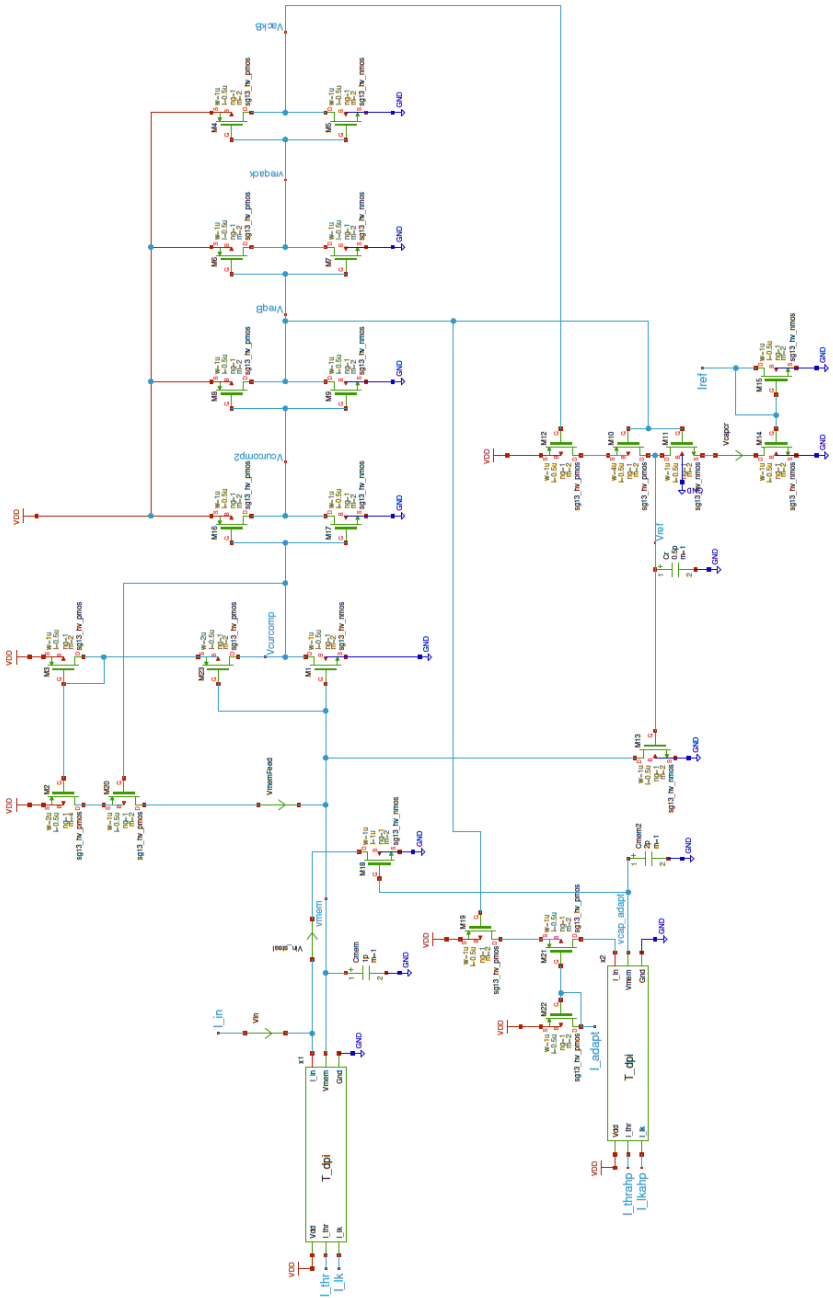
C Python script to plot `.raw` NGSPICE simulations

Listing 1 provides a minimal Python script for loading an `NGSPICE .raw` output file and visualizing selected waveforms using *plotly*. The script can be extended to include additional signals or customized post-processing.

```python
import matplotlib
import matplotlib.pyplot as plt
from spicelib import RawRead
import numpy as np

import plotly.io as pio
pio.renderers.default = "notebook"
import plotly.graph_objects as go

# Load the NGSPICE raw simulation file
raw = RawRead("./simulations/neuron_T.raw")

# Print all available signals contained in the raw file
print("Traces:", raw.get_trace_names())

# Extract the time vector from the simulation
t = raw.get_trace("time").get_wave()

# Extract relevant voltage waveforms
vmem = raw.get_trace("v(vmem)").get_wave()
vreq = raw.get_trace("v(vreq)").get_wave()

# Create an interactive Plotly figure
fig = go.Figure()

# Add each signal as a separate trace
fig.add_trace(go.Scatter(x=t, y=vmem, name="Vmem"))
fig.add_trace(go.Scatter(x=t, y=vreq, name="Vreq"))

# Configure plot appearance
fig.update_layout(
    title="neuron\\_T",
    xaxis_title="Time (s)",
    yaxis_title="Voltage (V)",
    template="plotly_white"
)

fig.show()
```

Listing 1: Python script for reading SPICE data and plotting signals

D Full Shematics



References

[1] S. H. Lee, "Neuromorphic mechanoreceptor chip using open-source tools," Master's thesis, Institute of Neuroinformatics, Universität Zürich and ETH Zürich, Zurich, Switzerland, Jun. 2025, m.Sc. Biomedical Engineering Research Project.

[2] A. K. Sharma, M. Madhusudan, S. M. Burns, P. Mukherjee, S. Yaldiz, R. Harjani, and S. S. Sapatnekar, "Common-centroid layouts for analog circuits: Advantages and limitations," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2021, pp. 1224–1229.

[3] M.-D. Ker and Z.-H. Jiang, "Overview on latch-up prevention in cmos integrated circuits by circuit solutions," *IEEE Journal of the Electron Devices Society*, vol. 11, pp. 141–152, 2023.

[4] J.-T. Chen, C.-Y. Lin, and M.-D. Ker, "On-chip esd protection device for high-speed i/o applications in cmos technology," *IEEE Transactions on Electron Devices*, vol. 64, no. 10, pp. 3979–3985, 2017.

[5] J. V. Faricelli, "Layout-dependent proximity effects in deep nanoscale cmos," in *IEEE Custom Integrated Circuits Conference 2010*, 2010, pp. 1–8.